

Ramas en Git

2. Ramas en Git

Qué es una rama

Las ramas permiten trabajar en una funcionalidad, arreglar un error o experimentar sin tocar el código que ya funciona. Sin ramas, cualquier cambio a medias quedaría mezclado con el código estable.

Una rama es una etiqueta que apunta a un commit

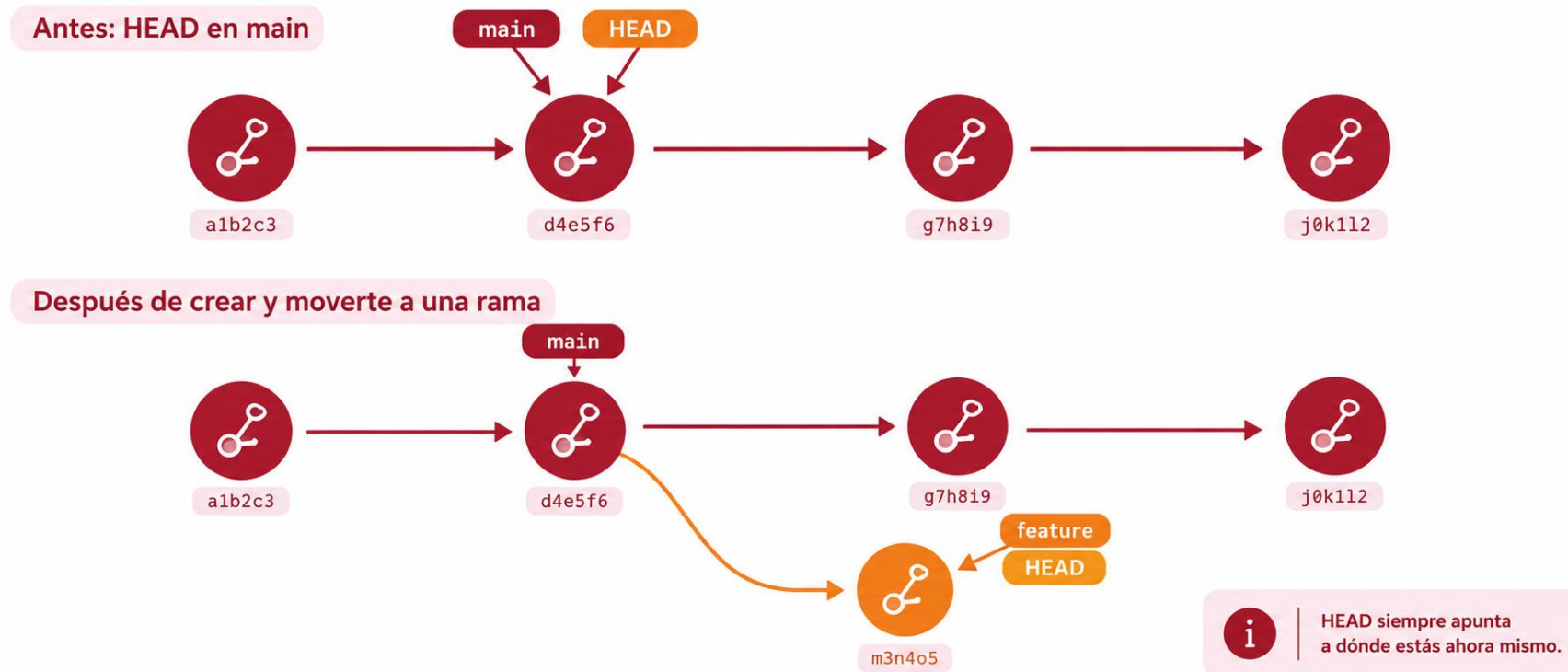
Avanza automáticamente cada vez que haces un commit estando en ella. La rama por defecto se llama main.

```
main:      A — B ————— C
           //
experimento:  \\ — Cambio experimental
```

main y experimento parten del mismo punto (Commit B) y cada una avanza por su lado sin pisarse.

¿Qué es HEAD?

HEAD es un puntero especial que indica en qué rama (y commit) estás trabajando ahora mismo. Lo verás en la salida de git log, git status y git lga.



Comandos básicos

Comando	Qué hace	Cuándo usarlo
<code>git branch</code>	Lista ramas	Para ver en cuál estás
<code>git branch <nombre></code>	Crea rama	Antes de empezar algo nuevo
<code>git switch <nombre></code>	Cambia de rama	Para moverte a otro contexto
<code>git switch -c <nombre></code>	Crea y cambia	Atajo para los dos pasos anteriores
<code>git branch -d <nombre></code>	Borra rama (seguro)	Tras fusionar
<code>git branch -D <nombre></code>	Borra rama (forzado)	Para descartar trabajo intencionalmente

`git switch`, no `git checkout`

`checkout` hacía demasiadas cosas distintas (cambiar de rama, recuperar archivos, moverse a un commit suelto). Git 2.23 separó `switch` (ramas) y `restore` (archivos): `switch` es más claro.

Visualizando el historial: el alias git lga

El comando completo es largo — se convierte en un alias, una sola vez

```
git log --graph --oneline --all --decorate
```

```
git config --global alias.lga "log --graph --oneline --all --decorate"
```

A partir de ahí: git lga

Te muestra un mapa visual de todas las ramas, en cualquier repositorio. Es el comando que más vas a usar para entender qué está pasando.

Historia de una rama: crear y moverte

Estás desarrollando una web y quieres experimentar con el color de fondo sin romper lo que ya funciona. Partes de main, con todo funcionando.

1-2. Crear la rama (no te mueve a ella)

```
git branch experimento-fondo
```

Misma posición todavía

main y experimento-fondo apuntan al mismo commit. Sigues en main.

3. Moverte a la nueva rama

```
git switch experimento-fondo
```

Switched to branch

git status muestra ahora On branch experimento-fondo.

4. Trabajar en la rama

```
git add estilos.css  
git commit -m "Fondo cambiado a rojo"
```

Historia de una rama: fusionar y limpiar

5-6. Volver a main y fusionar

```
git switch main  
git merge experimento-fondo
```

Fast-forward

main no ha avanzado mientras trabajabas: Git solo mueve la etiqueta main hasta donde está experimento-fondo. Ningún commit extra.

7. Limpieza

```
git branch -d experimento-fondo
```

El trabajo ya está en main

La rama ha cumplido su función. Se borra sin problema porque -d comprueba que ya está fusionada.

Antes de volver a main, si abres estilos.css el fondo rojo ha desaparecido: tus cambios están seguros en experimento-fondo, Git solo ha restaurado los archivos al estado de main.

Fusiones: fast-forward frente a 3-way merge

```
git switch main  
git merge nombre-de-la-rama
```

Fast-forward — historial lineal

main no ha avanzado desde que creaste tu rama. Git mueve el puntero hacia adelante. Historial lineal, sin commit extra.

3-way merge — commit de unión

main sí ha avanzado. Las líneas han divergido: Git busca el ancestro común, compara los cambios de cada lado y crea un merge commit que los une.

Cuando Git necesita crear un merge commit, abre tu editor de terminal (Vim o Nano) para que confirmes el mensaje.

Conflictos: cuándo ocurren

Un conflicto ocurre cuando las mismas líneas de un mismo archivo han sido modificadas de forma distinta en cada rama. Git no sabe cuál quedarse y para para que decidas tú.

```
<<<<<<< HEAD
Cambios en tu rama actual (main)
=====
Cambios en la rama que entra (nueva-rama)
>>>>>>> nueva-rama
```

1. Edita el archivo

Borra las marcas <<<, === y >>>, deja solo la versión final.

2. git add archivo

Añade el archivo resuelto al staging.

3. git commit

Cierra el merge con un commit.

Conflicto real: la misma línea, dos versiones

En main

```
titulo.txt: "Hola Planeta"
```

En cambio-titulo

```
titulo.txt: "Hola Universo"
```

git merge cambio-titulo → CONFLICT

La misma línea ha sido modificada de forma distinta en ambas ramas. Git detiene la fusión y marca el archivo afectado.

Si cada uno toca líneas distintas

Git las combina sin problema y no hay conflicto: el conflicto solo aparece cuando coincide la misma línea.

Al resolverlo (por ejemplo, dejando «Hola Planeta!»), el historial vuelve a unirse: git lga muestra el merge commit uniendo las dos líneas.

Ramas desde IntelliJ

No es obligatorio usar la terminal para todo

IntelliJ tiene integración completa con Git: puedes gestionar ramas desde la interfaz gráfica.

Barra inferior

Muestra la rama actual. Clic para ver todas las locales y remotas.

New Branch

Crear una rama nueva desde el menú.

Checkout

Cambiar de rama con un clic.

Los comandos que hace IntelliJ por debajo son exactamente los mismos que has practicado en terminal. Entenderlos ayuda a saber qué está pasando cuando algo falla en la GUI.

Ideas clave

1

`git branch <nombre>` crea una rama sin moverte a ella; `git switch <nombre>` te mueve.

2

`git switch -c <nombre>` crea y cambia en un solo paso.

3

HEAD apunta siempre a la rama y commit donde estás trabajando ahora.

4

`git merge <rama>` trae los cambios de esa rama a la rama donde estás.

5

Fast-forward: main no ha avanzado, Git mueve el puntero. Historial lineal.

6

Merge commit: ambas ramas han avanzado, Git crea un commit de unión.

7

Los conflictos ocurren cuando dos ramas tocan las mismas líneas del mismo archivo.

8

`git branch -d` borra si ya está fusionada; `-D` borra sin comprobar. `git log` muestra el grafo de ramas.